

How similar are two sequences?

Programming and Tools for AI – Week 7

James McDermott



Part 1: how similar are two sequences?

The problem

kitten

sitting

How many single-character edits turn one word into the other? An edit is an **insertion**, a **deletion** or a **substitution**. (The words in between don't have to be real words.)

The answer here is 3. Let's write an algorithm to find that.

How many AIs does it take to change a lightbulb?

That's a great question - and it cuts to the hert of the issue.

heart (x)

Why?

The same algorithm solves many problems:

- **Spelling.** The suggestions your editor offers are the dictionary words closest to what you typed.
- **Genetics.** DNA is a string over the alphabet ACGT, and two organisms are related to the extent that their sequences are close.
- **Music.** A folk melody is a list of integers. Melodies that are near each other are probably versions of the same tune, so the distances give us a family tree of who copied whom.
- **Machine learning.** Nearest-neighbour classification and kernel methods just use distance, so with today's concepts we can do classification on any sequence data, like DNA or music.

DNA

										10																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
--	--	--	--	--	--	--	--	--	--	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

from jalview.org

The obvious answer: Hamming distance

We line the two sequences up and count the positions where they disagree:

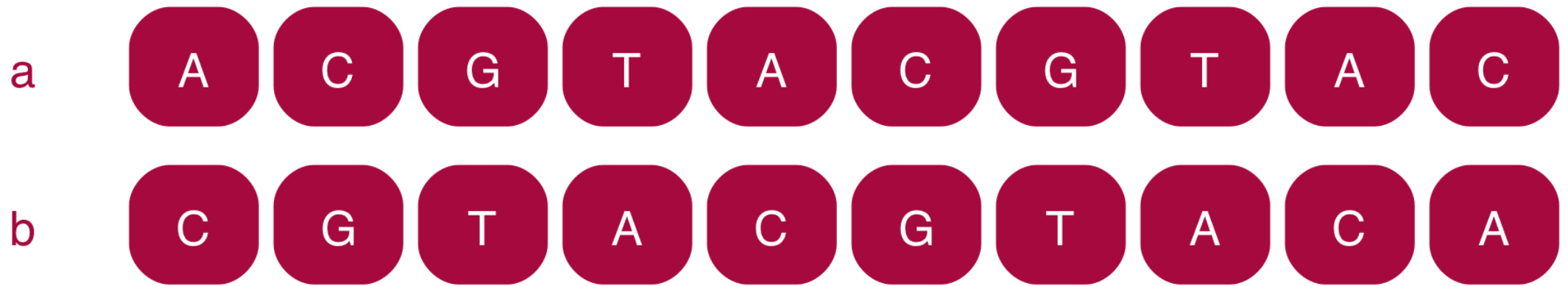
```
def hamming(a, b):  
    return sum(x != y for x, y in zip(a, b))
```

```
>>> hamming("ACGTACGTAC",  
            "ACGTTCGTAC")
```

1

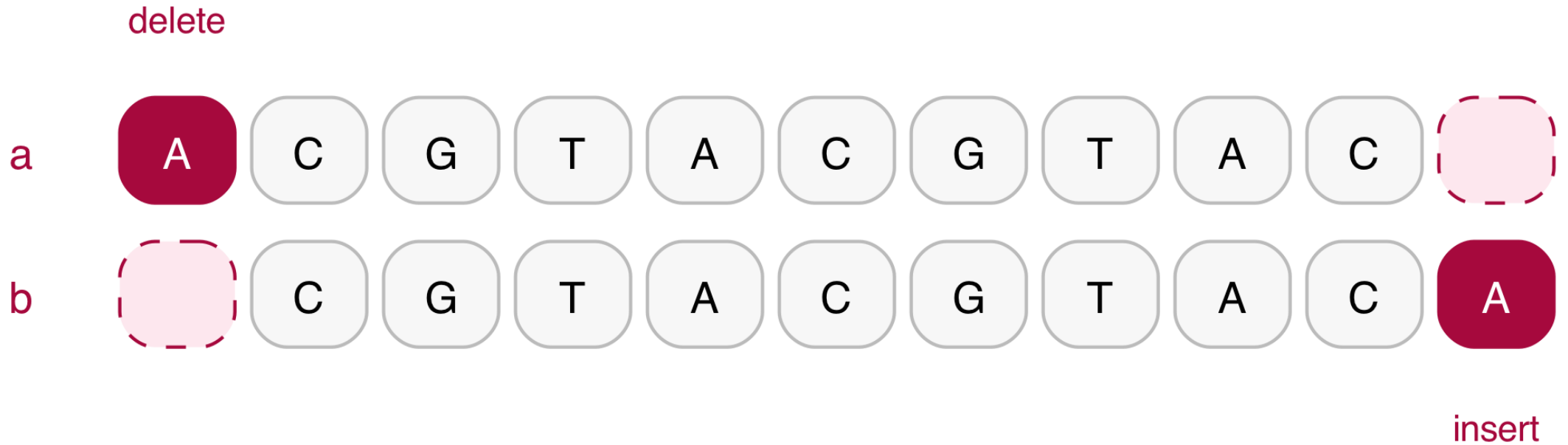
(What **type** is `x != y`?)

Where Hamming fails



`hamming("ACGTACGTAC", "CGTACGTACA") == 10`. Every position disagrees.

The same pair, shifted by one



Delete the leading A, then add an A at the end. The true distance is 2, but Hamming says 10.

The key difficulty is misalignment

Hamming assumes the two sequences are lined up, so it only ever sees **substitutions**. But in many applications, sequences are misaligned:

- a gene picks up an extra base and everything downstream shifts,
- a typist drops a letter,
- a singer holds one note for an extra beat.

A good distance method has to **find the alignment** as well as count the differences. It also has to work when the two lengths differ.

Edit distance, or Levenshtein distance

The smallest number of insertions, deletions and substitutions that turns a into b.

```
>>> levenshtein("kitten", "sitting")
```

```
3
```

```
>>> levenshtein("ACGTACGTAC", "CGTACGTACA")
```

```
2
```

```
>>> levenshtein("", "abc") # NB empty string case
```

```
3
```

```
>>> levenshtein([1, 2, 3], [2, 3]) # NB works on lists too
```

```
1
```

Named for Vladimir Levenshtein, who defined it in 1965.

The easy cases tell us where to start

Turning anything into the empty string means deleting every character, and building anything from the empty string means inserting every character:

```
levenshtein(a, "") == len(a)  
levenshtein("", b) == len(b)
```

Those two facts are the **base cases** of the algorithm we build in Part 2.

Think about the last character

Suppose we already know the answer (Levenshtein distance) for all **prefixes** of a and b . Then there are only three things the best solution can do with the final characters, when transforming a to b :

- **delete** $a[-1]$: pay 1, then turn $a[: -1]$ into b ,
- **insert** $b[-1]$ on the end of a : pay 1, then turn a into $b[: -1]$,
- **keep or substitute**, matching $a[-1]$ against $b[-1]$, and paying 1 if they differ. Then turn $a[: -1]$ into $b[: -1]$.

We try all three and take the smallest.

The recursive definition

```
def levenshtein(a, b):  
    if not a: # empty string  
        return len(b)  
    if not b: # empty string  
        return len(a)  
    cost = 0 if a[-1] == b[-1] else 1  
    return min(levenshtein(a[:-1], b) + 1, # delete  
               levenshtein(a, b[:-1]) + 1, # insert  
               # keep or substitute:  
               levenshtein(a[:-1], b[:-1]) + cost)
```

We have three recursive calls, each on a slightly smaller problem, and two base cases.

But unusably slow

```
>>> levenshtein("kitten", "sitting")
```

```
3                                     # 29,737 calls
```

```
>>> levenshtein("intention", "execution")
```

```
5                                     # 2,193,844 calls, 0.8 seconds
```

Each call makes three more, so the work grows like 3^n .

The definition is right, but there is a trick to make it faster.

Part 2: dynamic programming

The same trap in a simpler problem

```
def fib(n):  
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

```
>>> fib(30)
```

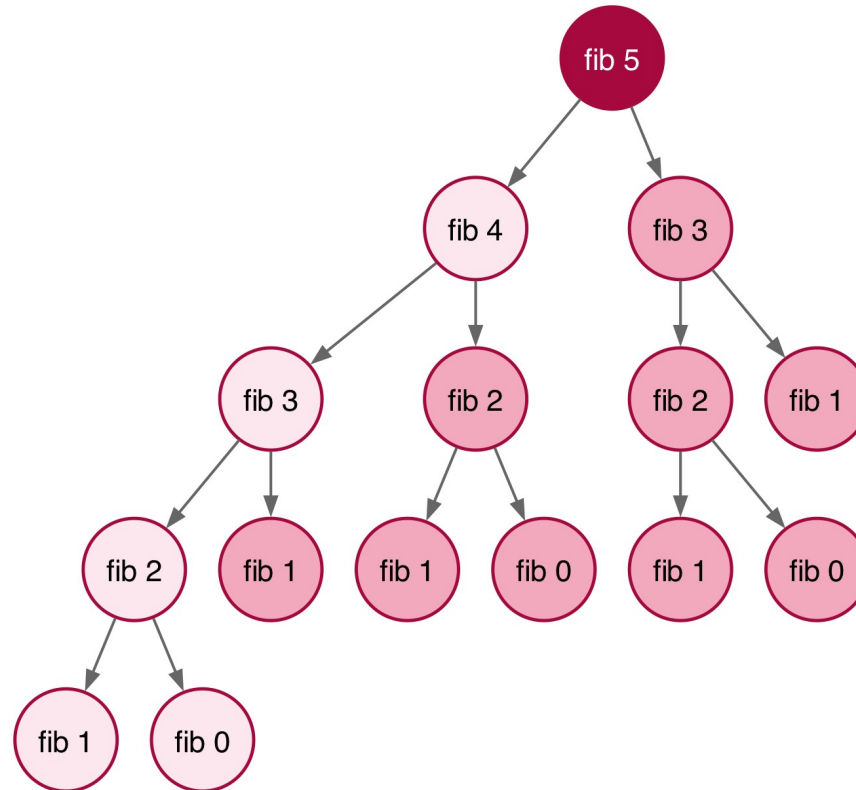
```
832040          # 2,692,537 calls, 0.5 seconds
```

```
>>> fib(35)
```

```
9227465         # 29,860,703 calls, 5 seconds
```

Every call makes two more, so this is 2^n work to produce an answer that a five-line loop gets in n steps.

Where the time goes



The call tree for `fib(5)`. Everything shaded is a repeat of something already computed: `fib(3)` is worked out twice, `fib(2)` three times, `fib(1)` five times. Deeper down it gets much worse.

Memoisation: remember the answers

We asked 30 million questions in `fib(35)`, but only 36 of them were unique.

```
from functools import cache
```

```
@cache
```

```
def fib(n):
```

```
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

```
>>> fib(35)
```

```
9227465          # 36 calls, 0.00003 seconds
```

By adding a cache, we go from 2^n to n . The decorator `@cache` wraps the function in a dict mapping arguments to results. On a repeat it returns the stored answer instead of recomputing.

Decorator syntax (not examinable)

```
@cache
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

is the same as:

```
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
fib = cache(fib) # NB cache is *higher-order*
```

Implementing cache

cache is implemented in functools similar to this:

```
def cache(f):  
    memo = {}  
    def cached_f(*args):  
        if args not in memo:  
            memo[args] = f(*args)  
        return memo[args]  
    return cached_f
```

cache takes a function and returns a function. The dict memo is created once when cache runs. It “stays alive” because cached_f refers to it. A function like cached_f is a **closure**.

*args collects the arguments into a tuple, and f(*args) spreads them back out as individual args.

Cache: assumptions and trade-offs

- Caching saves time but uses **memory**, one entry per distinct argument tuple.
- The arguments must be **hashable**, because they are stored as keys in a dict.
- The function must be **pure**: same arguments, same answer, no side effects (eg, does not read or write a file).

Top-down versus bottom-up

Memoisation works **top-down**: ask the original question, and fill the cache with answers to smaller sub-questions. But we know which questions there will be at each stage and which ones are needed to answer the next, so we can work **bottom-up** instead, with no recursion and no cache:

```
def fib(n):  
    F = [0, 1]  
    for i in range(2, n + 1):  
        F.append(F[i - 1] + F[i - 2])  
    return F[n]
```

Saving a little memory

Since we never look more than two back, we do not need the full list! We'll use this idea again later also. This might also remind you of the `iterate` function we saw earlier in the module.

```
def fib(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a
```

Memoising edit distance

We can use the same cache idea to make `levenshtein` fast. It works best if the cache keys are a tuple of `int` instead of tuple of `str`. The cached function is defined **inside** `levenshtein`, so the strings `a` and `b` are always available.

```
def levenshtein(a, b):  
    @cache  
    def d(i, j):  
        if i == 0:  
            return j  
        if j == 0:  
            return i  
        cost = 0 if a[i - 1] == b[j - 1] else 1  
        return min(d(i - 1, j) + 1,  
                   d(i, j - 1) + 1,  
                   d(i - 1, j - 1) + cost)  
    return d(len(a), len(b))
```

Turning the recursion around

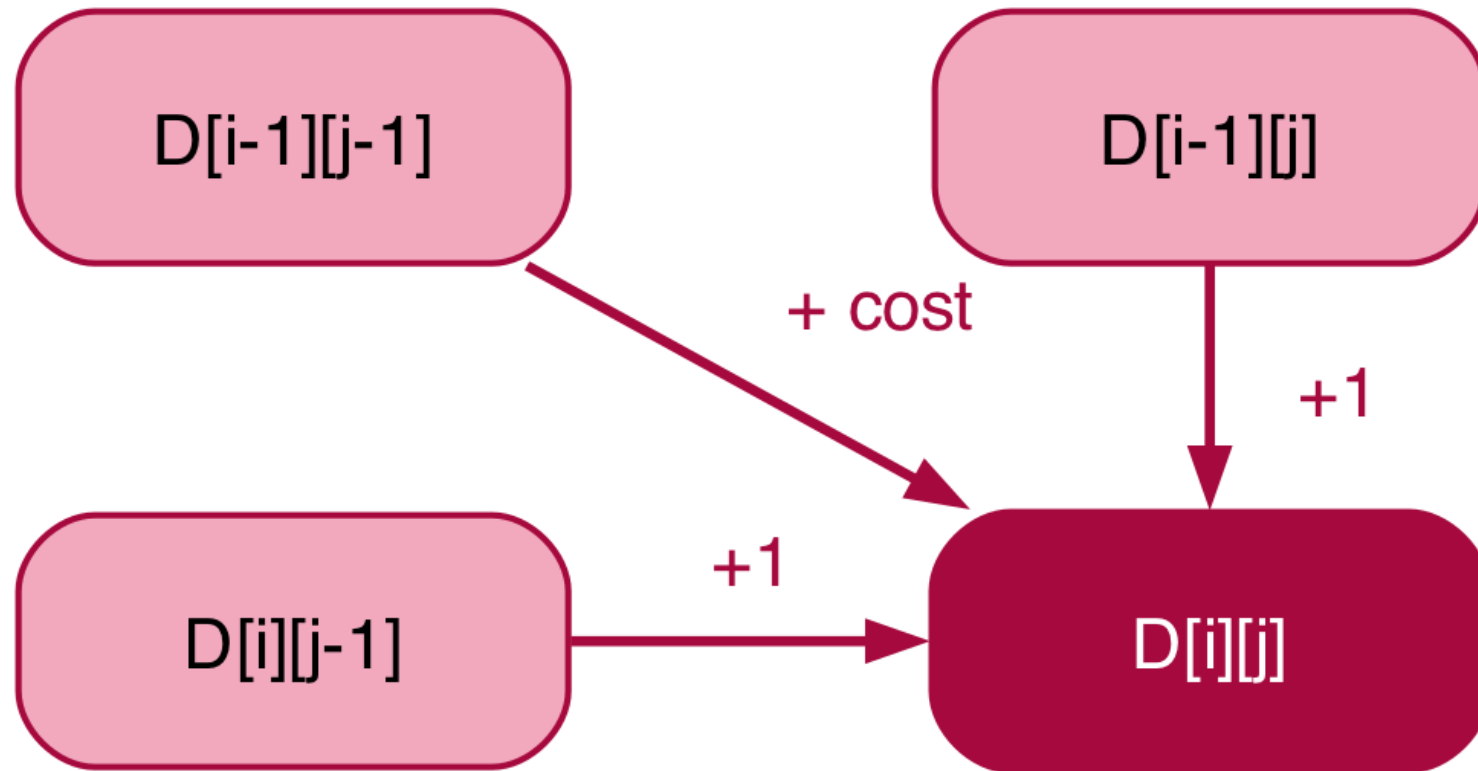
We are learning both top-down and bottom-up. There are problems where the bottom-up table is feasible and the top-down cached recursion is not, because it runs into a recursion-depth or a memory limit that the table doesn't.

Now we do for edit distance what we just did for Fibonacci. There are only $(n + 1) * (m + 1)$ distinct questions, and we know which depends on which. So we fill them in **bottom-up**.

Fibonacci subproblems are numbered by a single integer, so we created a list. Levenshtein subproblems are numbered by a **pair** (i, j) , (how much of a and how much of b) so we create a **grid**.

This bottom-up approach to recursive problems is called **dynamic programming**.

One grid cell as the min of three options



$D[i][j]$ is the edit distance between the first i characters of a and the first j characters of b . It depends on three neighbours: above, left, and above-left.

The base cases fill the edges

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1							
i	2							
t	3							
t	4							
e	5							
n	6							

Row 0 and column 0 are the easy cases: it takes j insertions to build the first j characters of `sitting` out of nothing, and i deletions to reduce the first i characters of `kitten` to nothing.

Then fill left to right, top to bottom

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2			
t								
e								
n								

Every cell needs only the three neighbours already computed, so one pass in reading order is enough. Here k, i, t against s, i, t, t costs 2.

The finished table, and the path back

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

The answer is always in the bottom-right. The shaded cells are the route that produced it: five diagonal steps, of which two cost 1 (k to s, e to i), then one step right to insert the g.

The code

```
def edit_table(a, b):
    D = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(len(a) + 1):
        D[i][0] = i                    # delete i characters
    for j in range(len(b) + 1):
        D[0][j] = j                    # insert j characters
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            cost = 0 if a[i - 1] == b[j - 1] else 1
            D[i][j] = min(D[i - 1][j] + 1,          # delete a[i-1]
                           D[i][j - 1] + 1,        # insert b[j-1]
                           D[i - 1][j - 1] + cost)  # keep or subs
    return D
def levenshtein(a, b): return edit_table(a, b)[-1][-1]
```

Two things to notice in that code

Off by one? D has $\text{len}(a) + 1$ rows because row 0 is the empty prefix. So row i is about $a[i - 1]$.

Build the table with a comprehension, not $*$.

```
D = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]    # correct
D = [[0] * (len(b) + 1)] * (len(a) + 1)              # WRONG
```

The wrong one would create $(\text{len}(a) + 1)$ references to a single list of zeros.

Part 3: Reading the answer; and space

Getting the edits back out

Sometimes we don't just want the number of edits, we want the edits themselves. We walk back from the bottom-right. At each cell we find which of the three neighbours it came from.

Getting the edits back out

```
def edits(a, b):
    D = edit_table(a, b) # the main work
    i, j, steps = len(a), len(b), []
    while i > 0 or j > 0:
        cost = 0 if i and j and a[i - 1] == b[j - 1] else 1
        if i and j and D[i][j] == D[i - 1][j - 1] + cost:
            if cost:
                steps.append(f"sub {a[i - 1]} -> {b[j - 1]}")
                i, j = i - 1, j - 1
            elif i and D[i][j] == D[i - 1][j] + 1:
                steps.append(f"del {a[i - 1]}")
                i -= 1
            else:
                steps.append(f"ins {b[j - 1]}")
                j -= 1
    steps.reverse(); return steps
```

The three edits

```
>>> edits("kitten", "sitting")  
['sub k -> s', 'sub e -> i', 'ins g']  
>>> edits("ACGTACGTAC", "CGTACGTACA")  
['del A', 'ins A']
```

Where several routes tie, the order of the tests decides which one we report. Any of them is a correct answer.

We did the same last week

	BFS	edit distance
forward pass	expand states, record came_from	fill D
what it leaves behind	one predecessor per state	one number per cell
backward pass	path_to	edits
what it recovers	the crossings	the three edits

Week 6's BFS did not just return True when it reached the goal: it filled a came_from dict as it went, and path_to walked backwards from the goal to recover the route. Both algorithms do all the work first and read the answer out backwards at the end.

What the table costs us

The table has $(n + 1) * (m + 1)$ cells. Filling a cell is three additions and a `min`, which is constant work.

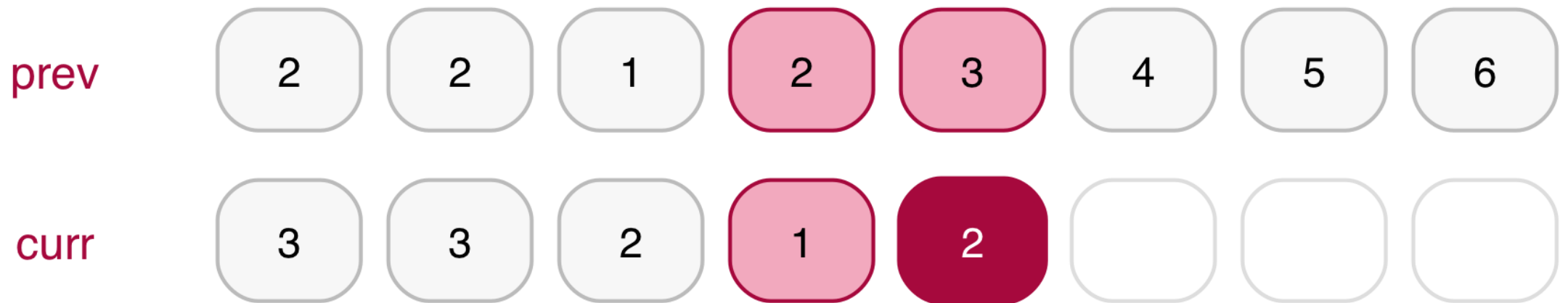
Time: $O(nm)$. Space: $O(nm)$.

The traceback is a second pass over a table we have already filled, so it costs $O(n + m)$ time which is negligible since we already have **$O(nm)$** .

Compare with where we started. The plain recursion was $O(3^n)$ in time, so we have improved the **computational complexity**.

But we can still reduce the memory by a lot more.

We only ever look one row back



$D[i][j]$ needs two cells from the row above and the one just written to its left. Nothing ever reaches two rows back, and nothing looks forward, so we do not need the table at all — only the **previous row** and the **row we are building**. Then `curr` becomes the new `prev` and the old one is thrown away.

Space: $O(m)$

The loop is the one we already have, with $D[i - 1]$ renamed `prev` and $D[i]$ renamed `curr`. The base cases are still there: `prev` starts as $[0, 1, 2, \dots]$, and each `curr` starts with the row number.

Time: $O(nm)$. Space: $O(m)$.

This is an exercise this week (a small change to given code).

What we gave up

The one-row version returns the same numerical result, and saves a lot of space.

However cannot tell you the edits. The traceback needs the whole table.

That is a typical trade-off.

If you need the alignment *and* linear space, Hirschberg's algorithm gets both, at twice the running time. **Not examinable.**

Which sequence should be the row?

In the two-row version, both `prev` and `curr` rows are as long as `b`, so the space is $O(\text{len}(b))$ — the **second** argument. Edit distance is symmetric, so we may swap the arguments freely and put the shorter sequence second:

```
if len(b) > len(a):  
    a, b = b, a           # keep the rows short
```

Space drops from $O(m)$ to $O(\min(n, m))$. If `n` and `m` are very different it is a large space saving.

The **time** does not change: it is $O(nm)$ either way, because `n * m` is the same product whichever way round we write it.

Three ways to run one recurrence

	time	space	gives the edits
plain recursion	$O(3^n)$	large call stack	yes
@cache	$O(nm)$	$O(nm)$ + dict overhead	yes
full table	$O(nm)$	$O(nm)$	yes
one row	$O(nm)$	$O(\min(n, m))$	no

Four different approaches with different time/space complexity. With two random 2000-character sequences: the cache takes 4.6s and 847 MB, the full table 2.3s and 167 MB, one row 1.7s and 9 MB. The cache also hits Python's recursion limit, although `sys.setrecursionlimit` can avoid that. (Not examinable.)

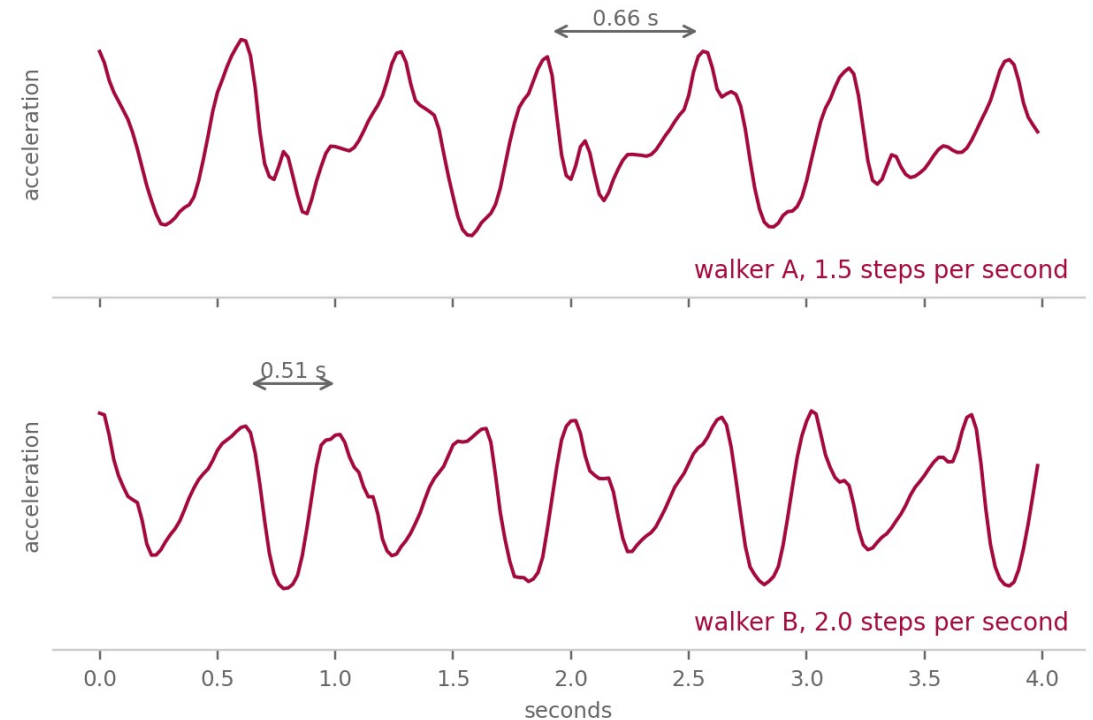
Part 4: dynamic time warping

The same problem, with numbers

A phone or a watch reports acceleration many times a second. Given a short recording, which human activity was it: walking, climbing stairs, or falling?

One approach is **nearest neighbour**. Keep a library of labelled recordings, and label a new one by whichever recording in the library it most resembles. Eg, **fall detection** for elderly people living alone works like this.

We need a distance between two lists of floats.



Two people walking, from the UCI Human Activity Recognition dataset.

Why the obvious distance fails

Two recordings of the same walk, one starting a quarter-second later:

```
>>> a = [0, 1, 3, 6, 3, 1, 0]
```

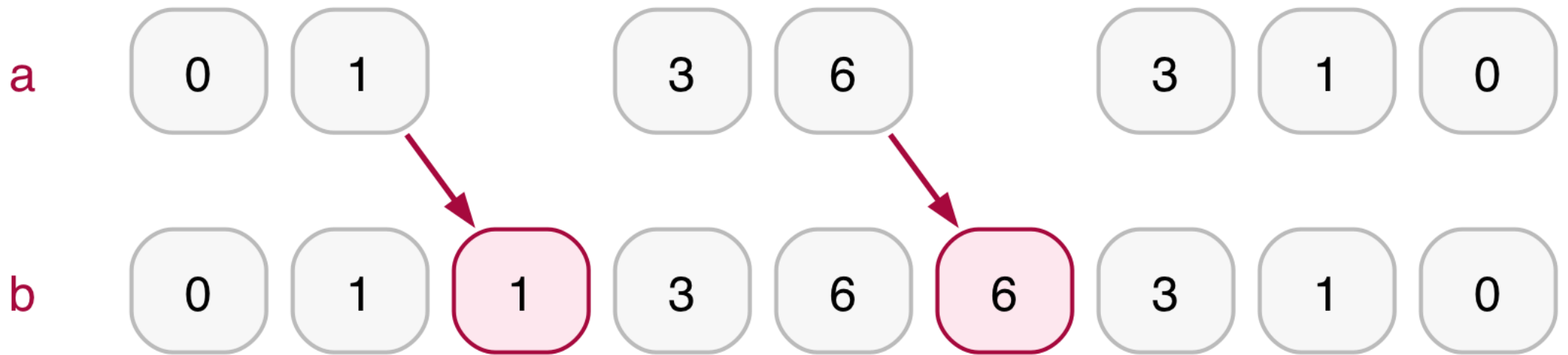
```
>>> b = [0, 0, 1, 3, 6, 3, 1]
```

```
>>> math.dist(a, b)
```

```
5.29
```

Euclidean distance is like Hamming distance for floats: it compares position `i` with position `i`. We saw the same misalignment problem for Hamming distance.

Stretch instead of substitute



b is the same shape, but stretched. We want a method that shows a and b are the same.

Dynamic time warping

We keep the DP table and three-way min. We change two things:

- an edit no longer costs 1 or 0, it costs $\text{abs}(a[i-1] - b[j-1])$, so nearly equal values are nearly free,
- there is no delete/insert, only **stretch**: going down or right means using one value twice.

Dynamic time warping

```
def dtw(a, b):  
    D = [[INF] * (len(b) + 1) for _ in range(len(a) + 1)]  
    D[0][0] = 0.0  
    for i in range(1, len(a) + 1):  
        for j in range(1, len(b) + 1):  
            cost = abs(a[i - 1] - b[j - 1])  
            D[i][j] = cost + min(D[i - 1][j],  
                                 D[i][j - 1],  
                                 D[i - 1][j - 1])  
    return D[-1][-1]
```

DTW examples

```
>>> # stretched
```

```
>>> dtw([0, 1, 3, 6, 3, 1, 0], [0, 1, 1, 3, 6, 6, 3, 1, 0])  
0.0
```

```
>>> # shifted
```

```
>>> dtw([0, 1, 3, 6, 3, 1, 0], [0, 0, 1, 3, 6, 3, 1])  
1.0
```

DTW = 0 for the stretched version, because it really is the same shape.

DTW = 1 for the shifted version, where Euclidean distance would say 5.29.

Different lengths are no longer a problem.

Levenshtein versus DTW

Levenshtein

$D[i][0] = i$
 $D[0][j] = j$

$\text{cost} = (0 \text{ if } a[i-1] == b[j-1] \\ \text{else } 1)$

$D[i][j] = \min($
 $D[i-1][j] + 1,$
 $D[i][j-1] + 1,$
 $D[i-1][j-1] + \text{cost})$

DTW

$D[i][0] = \text{INF}$
 $D[0][j] = \text{INF}$
 $D[0][0] = 0$

$\text{cost} = \text{abs}(a[i-1] - b[j-1])$

$D[i][j] = \min($
 $D[i-1][j] + \text{cost},$
 $D[i][j-1] + \text{cost},$
 $D[i-1][j-1] + \text{cost})$

Why the base cases differ

In Levenshtein, $D[i][0] = i$ means “you may delete the first i characters of a , for i units”.

DTW has no deletion. Every value in a must be matched to at least one value in b , and vice versa, so a path that skips the start of a sequence is not allowed at all. INF is how we say “this is not a legal path”. INF can never “win” in $\min$.

Making it cheaper

DTW is $O(nm)$ for time and space, like Levenshtein. The same one-row trick can be used to save space.

DTW also allows two approximations to save time and space.

- **Sakoe-Chiba band.** A good alignment rarely goes far from the diagonal, so we can refuse to fill cells more than w away from it. Then the algorithm is $O(nw)$. Even for long signals w is a small constant.
- **Downsample.** A hundred readings a second is far more resolution than “walking or falling” needs, so downsample to (say) 10.

Libraries

```
import Levenshtein # pip install Levenshtein  
Levenshtein.distance("kitten", "sitting")
```

```
from dtaidistance import dtw # pip install dtaidistance  
dtw.distance([0.1, 0.2, 0.3], [0.1, 0.12, 0.2, 0.29, 0.3])
```

These use C behind the scenes for speed.

Usually it's best to use the library. But we often use the concepts in other algorithms.

Summary

- Comparing two aligned sequences is simple: Hamming distance.
- If allowing **insertions and deletions**, we need **edit distance** (ED).
- The recursive definition of ED is slow, because it does repetitive work.
- **Memoisation** is a **top-down** technique for fast ED: remembering cases already done.
- **Dynamic programming** is fast ED **bottom-up**: filling a table starting from base cases.
- ED has time and space $O(nm)$.
- We can keep just two rows of the ED table to give space $O(\min(n, m))$.
- **DTW** is used for sequences of numbers rather than symbols. It uses the same table but a different cost per step.

New this week: `functools.cache` as a decorator; `*args`; a nested function with its own cache, closing over the enclosing scope; a 2D table as a list of lists; `enumerate(seq, start=1)`; `float("inf")`; `math.dist`.